

Requirement Analysis

Functional, non-functional, API and database requirements

Requirement Analysis Document

Project: FitBuddy — AI-Powered Personal Fitness Coach

1. Functional Requirements

FR-01: User Profile Input

- The system shall allow users to input their **Name**, **Age**, **Weight (kg)**, **Fitness Goal**, and **Workout Intensity**.
- Fitness Goals**: Weight Loss, Muscle Gain, General Wellness
- Intensity Levels**: Low, Medium, High
- Input validation must ensure all required fields are filled before submission.

FR-02: AI-Generated Workout Plan (Scenario 1)

- The system shall generate a **personalized 7-day workout plan** using the Google Gemini 2.0 Flash model.
- Each day must include: Focus area, Duration, Warm-up, Exercises (name, sets, reps, rest, tips), Cool-down, Motivational note.
- The plan must be returned in structured **JSON format**.
- Response must be displayed in an interactive, expandable day-card UI.

FR-03: Plan Update via Feedback (Scenario 2)

- The system shall allow users to submit **natural language feedback** to refine their existing plan.
- Users must provide their **User ID** (issued after plan generation) to update.

- The AI must incorporate the feedback and return a **modified 7-day plan**.
- Feedback examples: "More cardio", "Fewer rest days", "Focus on upper body".

FR-04: Nutrition & Recovery Tips (Scenario 3)

- The system shall provide **AI-generated nutrition and recovery tips** based on the user's fitness goal.
- Tips must include: Title, Summary, Foods to Include, Foods to Avoid, Hydration Advice, Recovery Advice.
- Optional: User ID connection to personalize the tip with the user's name.

FR-05: User History

- The system shall store all generated plans and nutrition tips in a **SQLite database**.
- Users can query their history via `GET /api/user/{id}/history`.

FR-06: Health Check API

- The system shall expose a `GET /api/health` endpoint returning the service status.

2. Non-Functional Requirements

NFR-01: Performance

- Workout plan generation must complete within **30 seconds** under normal conditions.
- API response time for non-AI endpoints must be under **500ms**.

NFR-02: Reliability

- The system must handle **AI JSON parse errors** gracefully and return meaningful error messages.
- Database connections must be properly opened and closed after every request.

NFR-03: Scalability

- The application must be structured so it can be migrated from SQLite to PostgreSQL without code restructuring.
- API endpoints must follow RESTful conventions for easy integration.

NFR-04: Security

- Gemini API key must be loaded from **environment variables**, not hardcoded.
- CORS must be configured to allow cross-origin requests.

NFR-05: Usability

- The UI must be **responsive** and usable on mobile, tablet, and desktop.
- The app must load in under **3 seconds** on a standard internet connection.
- All interactive elements must have appropriate **hover states and feedback**.

NFR-06: Maintainability

- Code must follow modular structure: separate `main.py` (backend), `app.js` (frontend logic), `style.css` (styling).
- All API routes must include **docstrings**.

3. API Requirements

Method	Endpoint	Request Body	Response
POST	<code>/api/generate-plan</code>	<code>{name, age, weight, goal, intensity}</code>	<code>{success, user_id, plan_id, plan}</code>
POST	<code>/api/update-plan</code>	<code>{user_id, feedback}</code>	<code>{success, plan_id, plan, feedback_applied}</code>
POST	<code>/api/nutrition-tip</code>	<code>{user_id, goal}</code>	<code>{success, nutrition}</code>
GET	<code>/api/user/{id}/history</code>	—	<code>{user, plans, nutrition_tips}</code>
GET	<code>/api/health</code>	—	<code>{status, service, version}</code>

4. Database Schema Requirements

Users Table

```
CREATE TABLE users ( id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL, age INTEGER, weight REAL, goal TEXT, intensity TEXT, created_at TEXT );
```

Workout Plans Table

```
CREATE TABLE workout_plans ( id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER, plan_json TEXT, feedback TEXT, created_at TEXT, FOREIGN KEY (user_id) REFERENCES users(id) );
```

Nutrition Tips Table

```
CREATE TABLE nutrition_tips ( id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER, goal TEXT, tip TEXT, created_at TEXT, FOREIGN KEY (user_id) REFERENCES users(id) );
```

5. UI/UX Requirements

Requirement	Description
Dark theme	Deep navy background with glassmorphism cards
Animated background	Floating gradient orbs for visual depth
Tab navigation	3 tabs: Generate Plan, Update Plan, Nutrition Tips
Responsive grid	CSS Grid adapting to screen size
Loading states	Progress bar + animated emoji during AI processing
Toast notifications	Success, error, and info toasts in top-right corner
Counter animations	Hero stats animate from 0 to target values on load
Expandable cards	Day cards expand on click to show full exercise details

6. Technology Stack

Component	Technology	Version
Backend Framework	FastAPI	0.115.5
ASGI Server	Uvicorn	0.32.1
AI Integration	Google Generative AI	0.8.3
Data Validation	Pydantic	2.10.3
Database	SQLite3	Built-in
File Upload	python-multipart	0.0.17
Async Files	aiofiles	24.1.0
Frontend	Vanilla HTML/CSS/JS	—
Fonts	Google Fonts (Outfit, Space Grotesk)	—

7. Constraints

- Must use **Google Gemini API** (not OpenAI or others)
- Frontend must be served by FastAPI (no separate server)
- Database must be **SQLite** for portability
- No frontend framework (plain HTML/CSS/JS only)
- API key must be configurable via environment variable